

# InRisk Labs – Full Stack Weather Explorer: Deep Analysis Report

**Author:** Case Study Analysis  
**Date:** August 10, 2026  
**Subject:** Full Stack Engineer – Climate-Risk Platform Case Study

## 1. Executive Summary

This case study requires building a **full-stack weather explorer** that simulates the core workflow of a climate-risk platform: **ingest** → **store** → **retrieve** → **visualize**. The application must fetch historical daily weather data from the Open-Meteo API, persist raw JSON in cloud object storage, and expose a web dashboard for end-to-end interaction.

The solution is evaluated across five dimensions: full-stack integration, backend engineering, frontend engineering & UI, code quality, and practicality & performance.

This report provides an in-depth analysis of the requirements, architecture decisions, technology choices, implementation strategy, deployment plan, and risk mitigation — serving as both a design document and a defense-ready reference for every technical choice.

## 2. Requirements Decomposition

### 2.1 Functional Requirements

| ID   | Requirement                                                          | Category      |
|------|----------------------------------------------------------------------|---------------|
| FR-1 | Fetch historical daily weather for user-chosen location + date range | Backend / API |

| ID    | Requirement                                                                     | Category        |
|-------|---------------------------------------------------------------------------------|-----------------|
| FR-2  | Store raw JSON in cloud object storage with deterministic naming                | Backend / Cloud |
| FR-3  | <code>POST /store-weather-data</code> – validate, fetch, store, return filename | Backend API     |
| FR-4  | <code>GET /list-weather-files</code> – list objects with name, size, created_at | Backend API     |
| FR-5  | <code>GET /weather-file-content/{file}</code> – return stored JSON or 404       | Backend API     |
| FR-6  | Input Panel with lat/lon/dates and Fetch & Store action                         | Frontend        |
| FR-7  | Stored Files browser with click-to-view                                         | Frontend        |
| FR-8  | Line chart of daily max/min temperature                                         | Frontend        |
| FR-9  | Paginated table (10/20/50 rows) of daily variables                              | Frontend        |
| FR-10 | Responsive layout (desktop/tablet/mobile)                                       | Frontend        |

## 2.2 Non-Functional Requirements

| ID    | Requirement                                                 | Category     |
|-------|-------------------------------------------------------------|--------------|
| NFR-1 | Proper validation with clear error messages (400/404/5xx)   | Backend      |
| NFR-2 | CORS enabled for frontend origin                            | Backend      |
| NFR-3 | Modular code structure (routes, storage client, validation) | Code Quality |
| NFR-4 | Efficient bucket listing (SDK methods, no brute scans)      | Performance  |
| NFR-5 | Loading/error states in all async operations                | Frontend UX  |

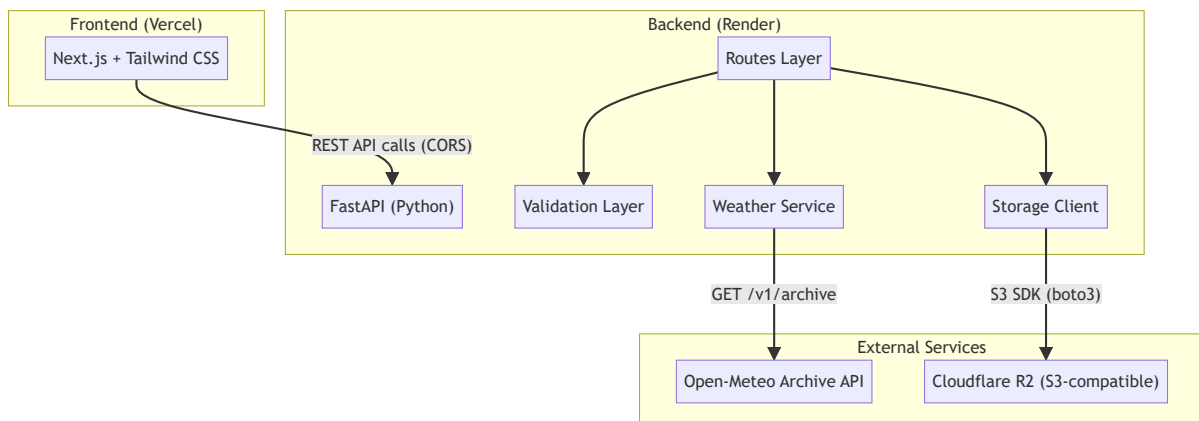
| ID    | Requirement                                                       | Category       |
|-------|-------------------------------------------------------------------|----------------|
| NFR-6 | Free-tier / no-cost cloud resources only                          | Infrastructure |
| NFR-7 | Live, accessible deployment URL at review time                    | Deployment     |
| NFR-8 | Clear commit history reflecting iterative development             | Process        |
| NFR-9 | Comprehensive README with setup instructions and design rationale | Documentation  |

## 2.3 Constraints

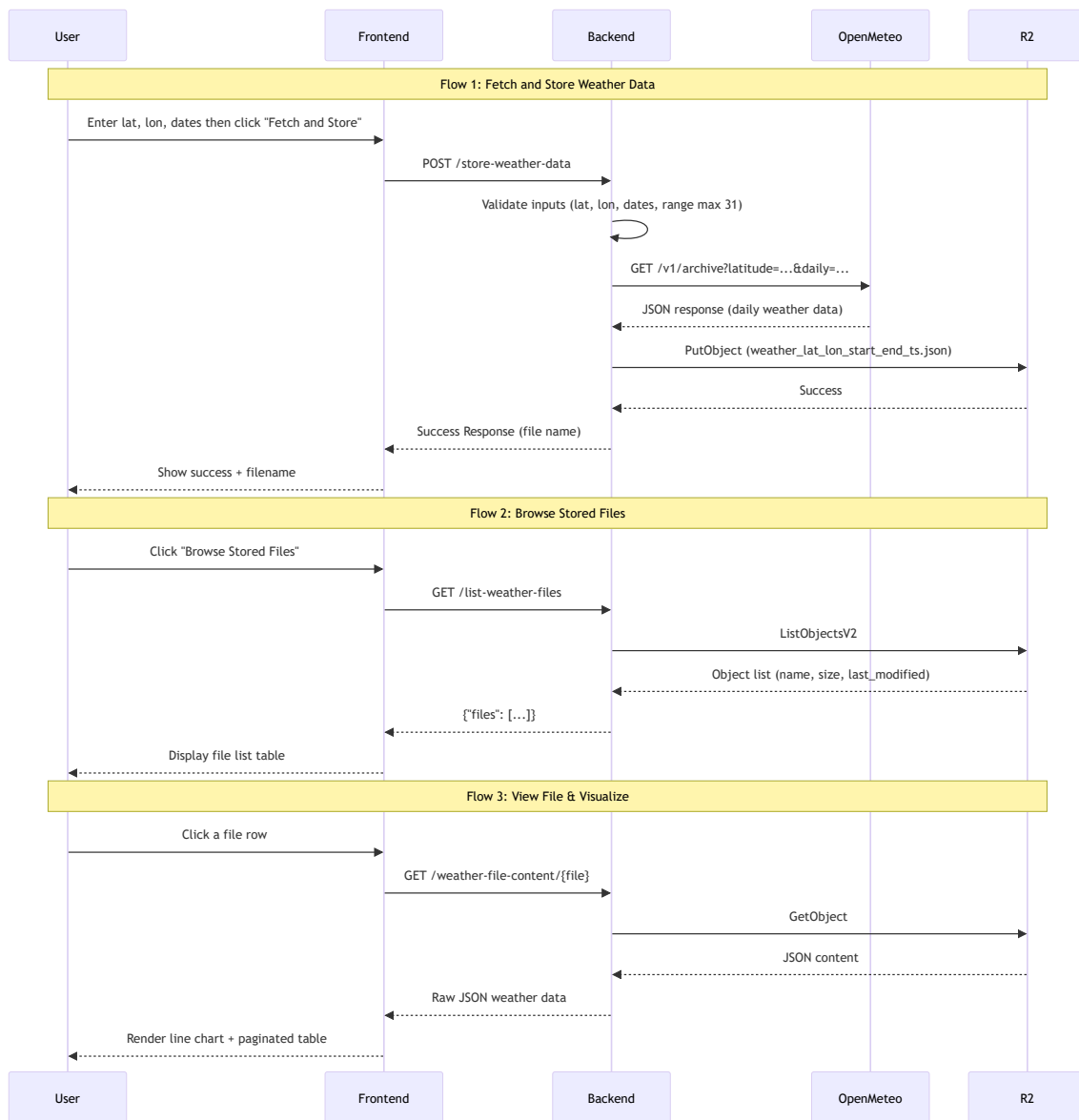
- **\*\*Date range** max 31 days per fetch
- **Latitude:**  $\in [-90, 90]$
- **Longitude:**  $\in [-180, 180]$
- **Weather variables (minimum):** `temperature_2m_max` , `temperature_2m_min` , `apparent_temperature_max` , `apparent_temperature_min`
- **File naming:** `weather_{lat}_{lon}_{start}_{end}_{timestamp}.json`
- **Free tier only:** No paid plans, no credit card commitment beyond provider free tier

## 3. Architecture Design

### 3.1 High-Level Architecture



## 3.2 Data Flow



## 3.3 Separation of Concerns

The architecture strictly separates:

1. **Frontend (Presentation):** React/Next.js handles all UI rendering, state management, and user interaction. No cloud SDK or direct API keys exposed.
2. **Backend (Business Logic):** FastAPI handles validation, orchestration, Open-Meteo calls, and cloud storage operations. Acts as a secure proxy.
3. **Storage (Persistence):** Cloudflare R2 stores raw JSON blobs. The backend is the only actor with storage credentials.

[!IMPORTANT] The backend acts as a **security boundary** — cloud storage credentials are never exposed to the frontend. All storage operations are proxied through authenticated backend endpoints.

## 4. Technology Stack Analysis & Justification

### 4.1 Backend: FastAPI (Python)

| Factor                | FastAPI                           | Flask                | Decision  |
|-----------------------|-----------------------------------|----------------------|-----------|
| <b>Async support</b>  | Native <code>async/await</code>   | Requires extensions  | FastAPI ✓ |
| <b>Validation</b>     | Built-in Pydantic models          | Manual / Flask-WTF   | FastAPI ✓ |
| <b>Auto-docs</b>      | Swagger UI + ReDoc out of box     | Requires Flask-Restx | FastAPI ✓ |
| <b>Type safety</b>    | Full type hints → auto-validation | No built-in          | FastAPI ✓ |
| <b>Performance</b>    | ASGI (Uvicorn) – high concurrency | WSGI – sequential    | FastAPI ✓ |
| <b>Learning curve</b> | Moderate                          | Low                  | Flask ✓   |

**Why FastAPI:** The case study values *proper validation, clear error messages, and correct HTTP status codes*. FastAPI's Pydantic integration auto-validates request bodies and generates meaningful 422 errors. Its async nature is ideal for I/O-bound workloads (external API calls + cloud storage operations). The auto-generated OpenAPI docs also serve as living API documentation.

### 4.2 Cloud Storage: Cloudflare R2

| Factor                      | Cloudflare R2           | AWS S3                 | GCS              |
|-----------------------------|-------------------------|------------------------|------------------|
| <b>Free tier storage</b>    | 10 GB/month (permanent) | 5 GB (12 months only)  | 5 GB (free tier) |
| <b>Free tier operations</b> | 1M writes, 10M reads/mo | 2,000 PUT, 20,000 GET  | 5,000 ops        |
| <b>Egress fees</b>          | <b>\$0 (unlimited)</b>  | \$0.09/GB after 100 GB | \$0.12/GB        |
| <b>Credit card required</b> | No CC for free tier     | CC required            | CC required      |
| <b>S3 compatibility</b>     | Full S3 API compatible  | Native                 | Different SDK    |
| <b>Free tier expiry</b>     | Never expires           | 12 months              | Always free tier |

**Why Cloudflare R2:** 1. **No credit card required** — directly satisfies the constraint *"nothing that requires a paid plan or credit card commitment beyond a provider's free tier."* 2. **Zero egress fees** — critical for a dashboard that repeatedly fetches stored JSON files for visualization. 3. **Permanent free tier** — the app won't break after a 12-month trial period. 4. **S3-compatible API** — allows using the battle-tested `boto3` library, making the code portable to actual AWS S3 if needed. 5. **10 GB storage** — more than sufficient for weather JSON files (each typically 1–5 KB).

[!NOTE] Using `boto3` with a custom endpoint URL means the storage client code is **provider-agnostic**. Switching to AWS S3 or MinIO requires only changing the endpoint URL and credentials — zero code changes in the storage abstraction layer.

### 4.3 Frontend: Next.js + Tailwind CSS

| Factor         | Next.js  | React (CRA/Vite) | Decision     |
|----------------|----------|------------------|--------------|
| <b>SSR/SSG</b> | Built-in | Requires setup   | Next.js<br>✓ |

| Factor            | Next.js                  | React (CRA/Vite)      | Decision     |
|-------------------|--------------------------|-----------------------|--------------|
| <b>Routing</b>    | File-based (automatic)   | Manual (React Router) | Next.js<br>✓ |
| <b>Deployment</b> | Vercel native (free)     | Needs separate host   | Next.js<br>✓ |
| <b>SEO</b>        | Built-in head management | React Helmet          | Next.js<br>✓ |
| <b>API routes</b> | Built-in (optional)      | None                  | Next.js<br>✓ |

**Why Next.js:** The case study explicitly lists "React or Next.js" and "Tailwind CSS." Next.js provides file-based routing, excellent Vercel deployment integration (free Hobby tier), and built-in optimization. Tailwind CSS is explicitly required by the case study specification.

#### 4.4 Charting: Recharts

| Factor                   | Recharts                        | Chart.js       | D3.js            |
|--------------------------|---------------------------------|----------------|------------------|
| <b>React integration</b> | Native React components         | Wrapper needed | Manual DOM       |
| <b>Bundle size</b>       | ~45 KB                          | ~65 KB         | ~70 KB           |
| <b>Ease of use</b>       | High (declarative)              | Medium         | Low (imperative) |
| <b>Responsive</b>        | Built-in<br>ResponsiveContainer | Plugin needed  | Manual           |

**Why Recharts:** It's a React-native charting library with declarative API. Its `ResponsiveContainer` component ensures charts resize correctly across breakpoints — directly addressing the responsive layout requirement.

#### 4.5 Backend Deployment: Render

| Factor                | Render                   | Fly.io        | Railway                |
|-----------------------|--------------------------|---------------|------------------------|
| <b>Free tier</b>      | Permanent                | Discontinued  | No permanent free tier |
| <b>Cold starts</b>    | ~30-60s after 15min idle | N/A           | N/A                    |
| <b>Docker support</b> | Yes                      | Yes           | Yes                    |
| <b>Auto-deploy</b>    | GitHub integration       | CLI           | GitHub integration     |
| <b>Cost</b>           | \$0                      | Pay-as-you-go | \$5/mo minimum         |

**Why Render:** It's the only major platform offering a **permanent free tier** for web services. The cold-start limitation is acceptable for a case study demo. The GitHub-connected auto-deploy pipeline provides clean commit-driven deployments.

## 4.6 Frontend Deployment: Vercel

Vercel's Hobby plan provides **100 GB bandwidth/month, 1M function invocations**, and **native Next.js support** — ideal for deploying the frontend at zero cost. The tight integration between Next.js and Vercel means zero configuration for deployment.

# 5. Backend Design (Detailed)

## 5.1 Project Structure

```

backend/
├── app/
│   ├── __init__.py
│   ├── main.py           # FastAPI app entry, CORS, lifespan
│   ├── config.py         # Environment variables, settings (Pydantic BaseS
│   └── routes/
│       ├── __init__.py
│       └── weather.py     # All three endpoint handlers
│   └── services/
│       └── __init__.py

```



```

| | | └─ storage.py          # R2/S3 client abstraction
| | |   └─ open_meteo.py     # Open-Meteo API client
| | └─ models/
| | |   └─ __init__.py
| | |   └─ schemas.py       # Pydantic request/response models
| | └─ utils/
| | |   └─ __init__.py
| | |   └─ validation.py    # Custom validators
└─ tests/
    |   └─ __init__.py
    |   └─ test_validation.py
    |   └─ test_weather_routes.py
    |   └─ test_storage.py
└─ requirements.txt
└─ Dockerfile
└─ .env.example
└─ render.yaml              # Render deployment config

```

[!TIP] This structure follows the **modular layered architecture** explicitly required by the case study: *"Simple, modular structure (routes, storage client, validation)."*

## 5.2 Input Validation (Pydantic Models)

```

from pydantic import BaseModel, field_validator
from datetime import date, timedelta

class WeatherRequest(BaseModel):
    latitude: float
    longitude: float
    start_date: date
    end_date: date

    @field_validator("latitude")
    @classmethod
    def validate_latitude(cls, v):
        if not -90 <= v <= 90:
            raise ValueError("Latitude must be between -90 and 90")
        return v

    @field_validator("longitude")
    @classmethod

```

```

def validate_longitude(cls, v):
    if not -180 <= v <= 180:
        raise ValueError("Longitude must be between -180 and 180")
    return v

@field_validator("end_date")
@classmethod
def validate_date_range(cls, v, info):
    start = info.data.get("start_date")
    if start and v < start:
        raise ValueError("end_date must be >= start_date")
    if start and (v - start).days > 31:
        raise ValueError("Date range max 31 days")
    return v

```

**Design Rationale:** Pydantic validators run automatically when FastAPI deserializes the request body. Invalid inputs produce HTTP 422 with structured error details — satisfying the "proper validation, clear error messages" requirement. We also add a custom exception handler to return HTTP 400 for business-rule violations (vs. 422 for malformed requests).

## 5.3 Open-Meteo Integration

```

OPEN_METEO_BASE_URL = "https://archive-api.open-meteo.com/v1/archive"

DAILY_VARIABLES = [
    "temperature_2m_max",
    "temperature_2m_min",
    "apparent_temperature_max",
    "apparent_temperature_min",
]

async def fetch_weather_data(lat: float, lon: float, start: str, end: str) -> d
    params = {
        "latitude": lat,
        "longitude": lon,
        "start_date": start,
        "end_date": end,
        "daily": ",".join(DAILY_VARIABLES),
        "timezone": "auto",
    }
    async with httpx.AsyncClient(timeout=30.0) as client:

```

```
response = await client.get(OPEN_METEO_BASE_URL, params=params)
response.raise_for_status()
return response.json()
```

**Key Decisions:** - **httpx (async):** Used instead of `requests` because FastAPI runs on an ASGI server. Synchronous `requests` would block the event loop. `httpx` provides an identical API with native `async/await`. - **timezone: "auto"**: Ensures date boundaries match the location's local timezone, producing more meaningful daily aggregates. - **Timeout of 30s:** Open-Meteo can be slow for large date ranges; 30s is generous but prevents infinite hangs. - **Variable selection:** The four required variables plus potential extras (precipitation, windspeed) provide comprehensive weather context for the dashboard visualization.

## 5.4 Storage Client Abstraction

```
import boto3
from botocore.config import Config

class StorageClient:
    def __init__(self, endpoint_url, access_key, secret_key, bucket_name):
        self.bucket_name = bucket_name
        self.s3 = boto3.client(
            "s3",
            endpoint_url=endpoint_url,
            aws_access_key_id=access_key,
            aws_secret_access_key=secret_key,
            config=Config(signature_version="s3v4"),
            region_name="auto",
        )

    def upload_json(self, filename: str, data: dict) -> str:
        self.s3.put_object(
            Bucket=self.bucket_name,
            Key=filename,
            Body=json.dumps(data, indent=2),
            ContentType="application/json",
        )
        return filename

    def list_files(self) -> list[dict]:
        response = self.s3.list_objects_v2(Bucket=self.bucket_name)
```

```

files = []
for obj in response.get("Contents", []):
    files.append({
        "name": obj["Key"],
        "size": obj["Size"],
        "created_at": obj["LastModified"].isoformat(),
    })
return files

```

```

def get_file(self, filename: str) -> dict | None:
    try:
        response = self.s3.get_object(Bucket=self.bucket_name, Key=filename)
        return json.loads(response["Body"].read().decode("utf-8"))
    except self.s3.exceptions.NoSuchKey:
        return None

```

**Design Rationale:**

- **Abstraction layer:** The `StorageClient` class encapsulates all S3/R2 operations. Switching cloud providers requires only changing constructor parameters — not modifying any route handler code.
- `list_objects_v2`: Directly satisfies "*Efficient bucket listing (SDK methods; avoid brute scans)*". This SDK method uses server-side listing with pagination support.
- **ContentType:** `application/json`: Ensures proper MIME type for stored files, enabling direct browser access if needed.
- **Error handling:** `NoSuchKey` exception maps directly to HTTP 404 — clean error propagation.

## 5.5 File Naming Convention

```

from datetime import datetime

def generate_filename(lat: float, lon: float, start: str, end: str) -> str:
    timestamp = datetime.utcnow().strftime("%Y%m%dT%H%M%SZ")
    return f"weather_{lat}_{lon}_{start}_{end}_{timestamp}.json"

```

**Example output:** `weather_52.52_13.41_2024-01-01_2024-01-31_20260810T101530Z.json`

The UTC timestamp suffix ensures **uniqueness** — the same location and date range can be fetched multiple times without overwriting previous data, which is important for a climate research workflow where data provenance matters.

## 5.6 CORS Configuration

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",      # Local dev
        "https://weather-explorer.vercel.app", # Production
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

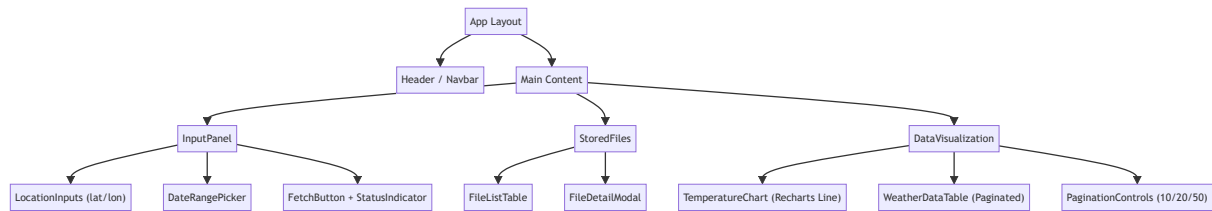
**Security Note:** Origins are explicitly allowlisted rather than using `allow_origins=["*"]`. This is a minor but meaningful security practice for a case study — it demonstrates awareness of CORS security implications.

## 5.7 Error Handling Strategy

| Scenario                 | HTTP Code | Response Body                                                                      |
|--------------------------|-----------|------------------------------------------------------------------------------------|
| Invalid lat/lon/dates    | 400       | <code>{"status": "error", "message": "Latitude must be between -90 and 90"}</code> |
| Malformed request body   | 422       | Pydantic auto-generated validation errors                                          |
| File not found in bucket | 404       | <code>{"status": "error", "message": "not found"}</code>                           |
| Open-Meteo API failure   | 502       | <code>{"status": "error", "message": "Weather API unavailable"}</code>             |
| R2/S3 failure            | 500       | <code>{"status": "error", "message": "Storage service error"}</code>               |
| Unexpected error         | 500       | <code>{"status": "error", "message": "Internal server error"}</code>               |

## 6. Frontend Design (Detailed)

### 6.1 Component Architecture



### 6.2 State Management

Given the moderate complexity (3 main features, no deep nesting), **React's built-in `useState` + `useReducer`** is sufficient. No external state library (Redux, Zustand) is needed.

| State                         | Location                                  | Type                  |
|-------------------------------|-------------------------------------------|-----------------------|
| Form inputs (lat, lon, dates) | <code>InputPanel</code> local state       | <code>useState</code> |
| Loading/error states          | Per-component local state                 | <code>useState</code> |
| File list                     | <code>StoredFiles</code> state            | <code>useState</code> |
| Selected file content         | Lifted to parent or context               | <code>useState</code> |
| Pagination (page, pageSize)   | <code>WeatherDataTable</code> local state | <code>useState</code> |

### 6.3 API Client Layer

```
// lib/api.ts
const API_BASE = process.env.NEXT_PUBLIC_API_URL;

export async function storeWeatherData(payload: WeatherRequest): Promise<StoreR> {
  const res = await fetch(`${API_BASE}/store-weather-data`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  if (!res.ok) {
```

```

    const error = await res.json();
    throw new ApiError(res.status, error.message || "Failed to store data");
  }
  return res.json();
}

export async function listWeatherFiles(): Promise<FileListResponse> {
  const res = await fetch(`${API_BASE}/list-weather-files`);
  if (!res.ok) throw new ApiError(res.status, "Failed to list files");
  return res.json();
}

export async function getWeatherFileContent(filename: string): Promise<WeatherD
  const res = await fetch(`${API_BASE}/weather-file-content/${encodeURIComponent(filename)}`);
  if (res.status === 404) throw new ApiError(404, "File not found");
  if (!res.ok) throw new ApiError(res.status, "Failed to fetch file");
  return res.json();
}

```

**Key Detail:** `encodeURIComponent(filename)` is critical because filenames contain dots, underscores, and colons — characters that could break URL parsing if unescaped.

## 6.4 UI/UX Design Specifications

---

### Input Panel

- **Latitude/Longitude:** Number inputs with `step="0.01"`, `min` / `max` constraints
- **Date pickers:** Native `<input type="date">` with `max` set to today (no future dates for historical API)
- **Submit button:** Shows spinner during loading, disables during fetch, displays success toast with filename
- **Client-side pre-validation:** Mirrors backend rules for instant feedback before network round-trip

### Stored Files

- **Table columns:** File Name, Size (formatted: KB/MB), Created At (relative time + tooltip with ISO)
- **Row click:** Loads file content and scrolls to visualization section
- **Empty state:** Illustrated empty state with "No files stored yet" message
- **Refresh button:** Manual refresh with last-updated timestamp

## Data Visualization

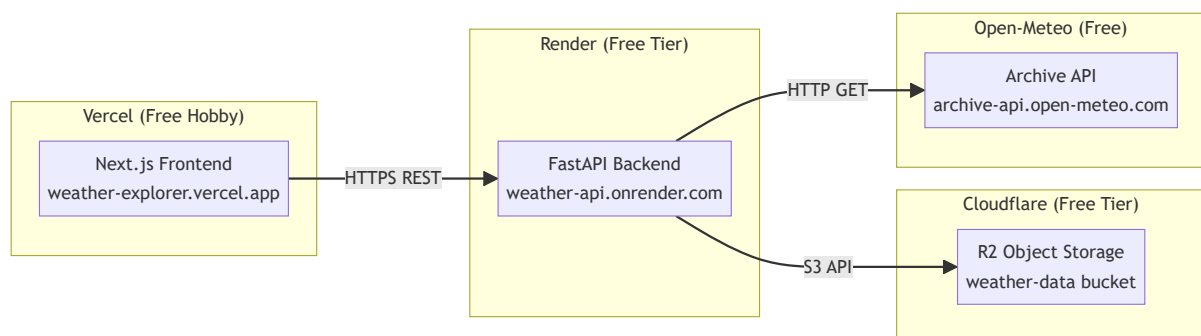
- **Line Chart:** Dual Y-axis or overlaid lines for `temperature_2m_max` (red/warm color) and `temperature_2m_min` (blue/cool color), with optional apparent temperature toggle
- **Interactive tooltips:** Show exact values on hover
- **Table:** All daily variables with sortable columns
- **Pagination controls:** Dropdown selector for 10/20/50 rows per page, Previous/Next buttons with page indicator

## 6.5 Responsive Breakpoints

| Breakpoint                              | Layout                                                            |
|-----------------------------------------|-------------------------------------------------------------------|
| <b>Desktop</b> ( $\geq 1024\text{px}$ ) | 2-column: Input Panel + Files side-by-side; Chart + Table below   |
| <b>Tablet</b> (768–1023px)              | Single column, all sections stacked, chart maintains aspect ratio |
| <b>Mobile</b> ( $< 768\text{px}$ )      | Single column, table scrolls horizontally, compact form layout    |

## 7. Deployment Strategy

### 7.1 Infrastructure Overview



### 7.2 Cost Analysis



| Service              | Component        | Free Tier Limit             | Expected Usage       | Headroom |
|----------------------|------------------|-----------------------------|----------------------|----------|
| <b>Vercel</b>        | Frontend hosting | 100 GB bandwidth/mo         | < 1 GB               | 99%+     |
| <b>Render</b>        | Backend hosting  | 750 hrs/mo (shared)         | ~720 hrs (always-on) | Adequate |
| <b>Cloudflare R2</b> | Object storage   | 10 GB, 1M writes, 10M reads | < 100 MB, < 1K ops   | 99%+     |
| <b>Open-Meteo</b>    | Weather API      | 10,000 calls/day            | < 100 calls/day      | 99%+     |

[!IMPORTANT] **Total cost: \$0/month.** All services operate well within free-tier limits. No credit card is required for Cloudflare R2 or Render. Vercel and Open-Meteo also offer free tiers without credit card requirements.

## 7.3 Environment Variables

### Backend (Render)

```
R2_ENDPOINT_URL=https://<account_id>.r2.cloudflarestorage.com
R2_ACCESS_KEY_ID=<cloudflare_r2_access_key>
R2_SECRET_ACCESS_KEY=<cloudflare_r2_secret_key>
R2_BUCKET_NAME=weather-data
ALLOWED_ORIGINS=https://weather-explorer.vercel.app
```

### Frontend (Vercel)

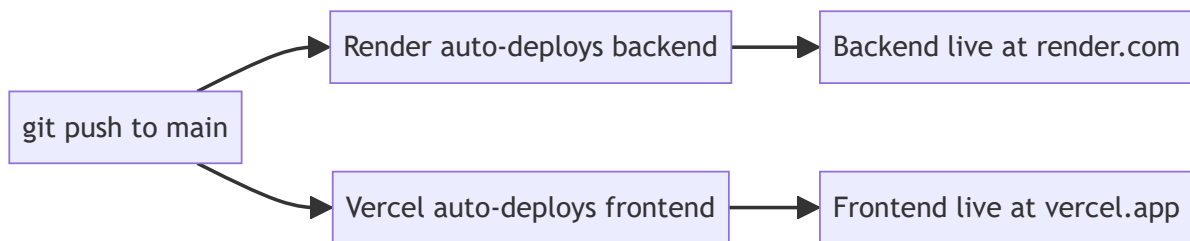
```
NEXT_PUBLIC_API_URL=https://weather-api.onrender.com
```

## 7.4 Render Cold Start Mitigation

Render's free tier spins down after 15 minutes of inactivity. First request after idle takes ~30-60 seconds. Mitigation strategies:

1. **Frontend UX:** Display a "Waking up server..." message if the first API call takes >5 seconds
2. **Health check endpoint:** `GET /health` returns `{"status": "ok"}` — frontend can ping this on page load to pre-warm the backend
3. **README transparency:** Document the cold-start behavior and expected warm-up time

## 7.5 CI/CD Pipeline



Both Render and Vercel support GitHub-connected auto-deployments. Every push to `main` triggers: 1. Render: Builds Docker image → deploys new container 2. Vercel: Builds Next.js → deploys to CDN edge

## 8. Security Considerations

| Risk                                                               | Mitigation                                                                                                                              |
|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>R2 credentials in frontend</b>                                  | Credentials are server-side only (backend environment variables); frontend never accesses R2 directly                                   |
| <b>CORS bypass</b>                                                 | Explicit origin allowlist; no wildcard <code>*</code> in production                                                                     |
| <b>Path traversal in <code>/weather-file-content/{file}</code></b> | Validate filename format (regex match against <code>weather_*</code> pattern); reject <code>/</code> , <code>..</code> , <code>\</code> |
| <b>API abuse (excessive fetches)</b>                               | Rate limiting via <code>slowapi</code> or middleware (e.g., 10 req/min per IP)                                                          |
| <b>Injection via lat/lon in filename</b>                           | Lat/Lon are validated as floats by Pydantic; no string injection possible                                                               |

| Risk                              | Mitigation                                                           |
|-----------------------------------|----------------------------------------------------------------------|
| <b>Open-Meteo API key leakage</b> | Open-Meteo doesn't require API keys for free tier; no secret to leak |

## 9. Testing Strategy

### 9.1 Unit Tests

```
# tests/test_validation.py
def test_valid_latitude():
    req = WeatherRequest(latitude=52.52, longitude=13.41,
                        start_date="2024-01-01", end_date="2024-01-07")
    assert req.latitude == 52.52

def test_invalid_latitude():
    with pytest.raises(ValidationError):
        WeatherRequest(latitude=91, longitude=0,
                        start_date="2024-01-01", end_date="2024-01-07")

def test_date_range_max_31_days():
    with pytest.raises(ValidationError):
        WeatherRequest(latitude=0, longitude=0,
                        start_date="2024-01-01", end_date="2024-03-01")

def test_end_date_before_start():
    with pytest.raises(ValidationError):
        WeatherRequest(latitude=0, longitude=0,
                        start_date="2024-01-10", end_date="2024-01-01")
```

### 9.2 Integration Tests

```
# tests/test_weather_routes.py
@pytest.fixture
def client():
    return TestClient(app)

def test_store_weather_data_success(client, mock_open_meteo, mock_r2):
    response = client.post("/store-weather-data", json={
```

```
        "latitude": 52.52, "longitude": 13.41,
        "start_date": "2024-01-01", "end_date": "2024-01-07"
    })
    assert response.status_code == 200
    assert response.json()["status"] == "ok"
    assert "file" in response.json()

def test_file_not_found_returns_404(client, mock_r2_empty):
    response = client.get("/weather-file-content/nonexistent.json")
    assert response.status_code == 404
    assert response.json()["status"] == "error"
```

### 9.3 Test Coverage Targets

| Layer          | Target | Approach                               |
|----------------|--------|----------------------------------------|
| Validation     | 100%   | Unit tests for all boundary conditions |
| Routes         | >80%   | Integration tests with mocked services |
| Storage client | >70%   | Unit tests with mocked boto3           |
| Frontend       | Manual | Browser testing + responsive checks    |

## 10. Edge Cases & Error Handling

| Edge Case                              | Expected Behavior                 |
|----------------------------------------|-----------------------------------|
| Latitude exactly at boundary (90, -90) | Accepted — inclusive range        |
| Longitude at boundary (180, -180)      | Accepted — inclusive range        |
| Start date = end date (1-day range)    | Accepted — valid single-day query |
| Date range max 31 days                 | Accepted — boundary value         |
| Date range 32 days                     | Rejected — 400 error              |

| Edge Case                                     | Expected Behavior                                            |
|-----------------------------------------------|--------------------------------------------------------------|
| Future dates                                  | API returns empty data; store empty result with note         |
| Non-existent location (e.g., middle of ocean) | Open-Meteo still returns data (nearest grid point); accepted |
| Empty bucket (no files)                       | Return <code>{"files": []}</code> — not an error             |
| File with special characters in name          | URL-encoded by frontend; decoded by backend                  |
| Open-Meteo API timeout                        | Return 502 with retry suggestion                             |
| R2 storage quota exceeded                     | Return 500 with descriptive message                          |
| Concurrent requests for same location/dates   | Both succeed; unique timestamps prevent collisions           |
| Very small lat/lon values (e.g., 0.001)       | Accepted — valid float values                                |

## 11. Performance Optimization

### 11.1 Backend

1. **Async I/O:** All external calls (Open-Meteo, R2) use async clients. Multiple concurrent requests don't block each other.
2. **Connection pooling:** `boto3` client is initialized once at app startup (singleton pattern), not per-request.
3. **Efficient listing:** `list_objects_v2` with optional `Prefix` filter for future enhancements.
4. **JSON streaming:** For large files, consider streaming response instead of loading entire JSON into memory.

### 11.2 Frontend

1. **SWR/React Query:** Use `useSWR` or `@tanstack/react-query` for data fetching with automatic caching, revalidation, and deduplication.

2. **Virtualized table:** For 31-day datasets, standard DOM rendering is fine. Pagination (10/20/50 rows) keeps DOM size small.
  3. **Lazy chart rendering:** Only render the chart component when data is loaded (code-split with `next/dynamic` ).
  4. **Debounced inputs:** Prevent rapid re-fetches while user is still typing coordinates.
- 

## 12. Repository Structure

---

```
weather-explorer/  
├── README.md                # Setup, architecture, design decisions  
├── .gitignore  
├── backend/  
│   ├── app/  
│   │   ├── main.py  
│   │   ├── config.py  
│   │   ├── routes/weather.py  
│   │   ├── services/storage.py  
│   │   ├── services/open_meteo.py  
│   │   ├── models/schemas.py  
│   │   └── utils/validation.py  
│   ├── tests/  
│   ├── requirements.txt  
│   ├── Dockerfile  
│   └── render.yaml  
├── frontend/  
│   ├── src/  
│   │   ├── app/  
│   │   │   ├── layout.tsx  
│   │   │   ├── page.tsx  
│   │   │   └── globals.css  
│   │   ├── components/  
│   │   │   ├── InputPanel.tsx  
│   │   │   ├── StoredFiles.tsx  
│   │   │   ├── TemperatureChart.tsx  
│   │   │   ├── WeatherDataTable.tsx  
│   │   │   └── PaginationControls.tsx  
│   │   └── lib/  
│   │       ├── api.ts  
│   │       └── types.ts  
│   └── tailwind.config.ts
```

```

|   |─ next.config.js
|   |─ package.json
|   |─ vercel.json
|   └─ docs/
|       └─ architecture.md           # Additional design documentation

```

## 13. README.md Outline

The README must demonstrate **the candidate's own thinking**. Recommended structure:

```

# Weather Explorer 🌤️

## Overview
Brief description of the project and its purpose.

## Live Demo
- **Frontend:** https://weather-explorer.vercel.app
- **Backend API:** https://weather-api.onrender.com/docs
- **Last Verified Live:** [date]
- **Cold Start Note:** Backend may take ~30s to wake up on first request.

## Architecture
High-level diagram and explanation of the three-tier architecture.

## Design Decisions & Assumptions
1. **Why Cloudflare R2 over AWS S3/GCS** – No credit card, zero egress, permanent
2. **Why FastAPI over Flask** – Built-in validation, async support, auto-docs
3. **Why Recharts** – Native React integration, responsive container
4. **Timezone handling** – Using `timezone=auto` for location-aware daily bounds
5. **File naming uniqueness** – UTC timestamp suffix prevents overwrites

## Tech Stack
| Layer | Technology | Rationale |
|-----|-----|-----|
| Backend | FastAPI + Python 3.11 | Async, Pydantic validation |
| Storage | Cloudflare R2 | Free, S3-compatible, zero egress |
| Frontend | Next.js 14 + Tailwind CSS | SSR, file routing, Vercel native |
| Charts | Recharts | React-native, responsive |
| Backend Host | Render | Free tier, Docker support |
| Frontend Host | Vercel | Free tier, Next.js native |

```

```
## Setup Instructions
### Prerequisites
- Python 3.11+, Node.js 18+, Cloudflare R2 bucket

### Backend
```bash
cd backend
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # Fill in R2 credentials
uvicorn app.main:app --reload
```

## Frontend

---

```
cd frontend
npm install
cp .env.example .env.local # Set API URL
npm run dev
```

## API Documentation

---

Interactive docs at `/docs` (Swagger UI) and `/redoc` (ReDoc).

## Testing

---

```
cd backend && pytest -v
```

## Trade-offs & Known Limitations

---

- Render cold starts (~30s after 15min idle)
- No authentication (acceptable for case study scope)
- No server-side caching of file lists ``

## 14. Evaluation Criteria Mapping

---



This section maps every evaluation criterion to the specific design decisions that address it:

## 14.1 Full-Stack Integration

| Criterion                        | How Addressed                                                                                |
|----------------------------------|----------------------------------------------------------------------------------------------|
| Frontend correctly calls backend | Dedicated API client layer ( <code>lib/api.ts</code> ); environment-variable-driven base URL |
| End-to-end flow                  | Sequential UX: Input → Fetch/Store → File appears in list → Click to visualize               |

## 14.2 Backend Engineering

| Criterion                                    | How Addressed                                                                                                                                                            |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Correct Open-Meteo usage                     | <code>/v1/archive</code> endpoint with all 4 required daily variables + <code>timezone=auto</code>                                                                       |
| Proper cloud bucket storage/retrieval/naming | <code>StorageClient</code> abstraction;<br><code>put_object</code> / <code>get_object</code> / <code>list_objects_v2</code> ;<br>deterministic naming with UTC timestamp |
| Clear validation and error handling          | Pydantic models with custom validators; exception handlers for 400/404/502/500                                                                                           |

## 14.3 Frontend Engineering & UI

| Criterion                      | How Addressed                                                                                                   |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Clean, responsive Tailwind UI  | Tailwind responsive utilities ( <code>sm:</code> , <code>md:</code> , <code>lg:</code> ); mobile-first approach |
| Useful chart + paginated table | Recharts <code>LineChart</code> with tooltips; paginated table with 10/20/50 selector                           |
| Good loading/error states      | Per-component loading spinners, error banners, toast notifications                                              |

## 14.4 Code Quality

| Criterion                          | How Addressed                                                                                       |
|------------------------------------|-----------------------------------------------------------------------------------------------------|
| Structure, readability, modularity | Layered architecture: routes → services → models; single responsibility                             |
| Sensible abstractions              | <code>StorageClient</code> class (provider-agnostic), <code>WeatherService</code> (API abstraction) |
| Testability                        | Dependency injection via constructor; mockable service boundaries                                   |

## 14.5 Practicality & Performance

| Criterion                   | How Addressed                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------|
| Efficient API usage         | Single Open-Meteo call per fetch; no redundant calls                                    |
| Efficient bucket operations | <code>list_objects_v2</code> (SDK method); no brute-force scanning                      |
| ≤31-day fetch limit         | Enforced in Pydantic validator; clear error message                                     |
| Empty/edge cases            | Empty bucket returns <code>[]</code> ; boundary values accepted; path traversal blocked |

## 15. Implementation Timeline

| Phase                           | Duration | Deliverables                                                                 |
|---------------------------------|----------|------------------------------------------------------------------------------|
| <b>Phase 1: Setup</b>           | 2 hours  | Repository, project scaffolding, R2 bucket, Render/Vercel accounts           |
| <b>Phase 2: Backend Core</b>    | 4 hours  | FastAPI app, validation, Open-Meteo service, storage client, all 3 endpoints |
| <b>Phase 3: Backend Testing</b> | 2 hours  | Unit tests, integration tests, edge case tests                               |

| Phase                         | Duration         | Deliverables                                                         |
|-------------------------------|------------------|----------------------------------------------------------------------|
| <b>Phase 4: Frontend Core</b> | 4 hours          | Next.js app, InputPanel, StoredFiles, API client                     |
| <b>Phase 5: Visualization</b> | 3 hours          | Recharts line chart, paginated table, responsive layout              |
| <b>Phase 6: Polish</b>        | 2 hours          | Loading states, error handling, empty states, responsive refinements |
| <b>Phase 7: Deployment</b>    | 2 hours          | Dockerize backend, deploy to Render + Vercel, verify live URLs       |
| <b>Phase 8: Documentation</b> | 1 hour           | README, architecture doc, commit history cleanup                     |
| <b>Total</b>                  | <b>~20 hours</b> | Full-stack app, deployed, documented, tested                         |

## 16. Risk Assessment

| Risk                                  | Likelihood | Impact | Mitigation                                           |
|---------------------------------------|------------|--------|------------------------------------------------------|
| Render free tier discontinued         | Low        | High   | Dockerized app; can migrate to any container host    |
| Cloudflare R2 free tier policy change | Low        | High   | S3-compatible code; swap endpoint to AWS/MinIO       |
| Open-Meteo API downtime               | Medium     | Medium | Cache last-known responses; graceful error messages  |
| Render cold start at review time      | High       | Medium | Health-check ping on page load; documented in README |
| Vercel Hobby plan limits exceeded     | Very Low   | Low    | Well within limits for case study traffic            |

## 17. Conclusion

---

This design delivers a **production-quality architecture** within free-tier constraints.

Key strengths:

1. **Zero-cost infrastructure** with Cloudflare R2 (no CC), Render (free), and Vercel (free) — totaling \$0/month
2. **Provider-agnostic storage abstraction** — the `StorageClient` class uses S3-compatible APIs, making migration trivial
3. **Robust validation** — Pydantic enforces all business rules at the request boundary with clear, actionable error messages
4. **Clean separation of concerns** — routes, services, and models are independently testable
5. **Thoughtful UX** — loading states, error handling, responsive design, and cold-start mitigation demonstrate attention to end-user experience

The commit history should reflect iterative development: infrastructure setup → backend core → tests → frontend shell → visualization → polish → deployment. Each commit should be a meaningful, reviewable unit of work.

# Premium Upgrade Analysis: InRisk Labs Weather Explorer 2.0

---

**Subject:** Architectural & UX Analysis for Top-Tier Climate Platform Upgrades **Target:** Enterprise-grade UI/UX and advanced analytical features

---

## 1. Vision & Strategy

---

The goal is to elevate the current functional MVP into a **"Top-Level" Climate Risk Dashboard**. This requires transitioning from a basic CRUD/fetching interface to an interactive, highly polished, and intelligent data exploration platform.

The upgrade focuses on two pillars: 1. **Sensory & UX Quality (The "Wow" Factor):** Micro-animations, bento-grid layouts, skeleton loaders, and premium dark-mode

glassmorphism. 2. **Frictionless Intelligence (Features):** Removing manual data entry (lat/lon) in favor of semantic search, adding automated analytical insights, and enabling data portability.

---

## 2. Top-Level UI Architecture (Next.js + Tailwind + Framer Motion)

---

To achieve a state-of-the-art aesthetic, we will overhaul the frontend component architecture:

### 2.1 Aesthetic System

---

- **Theme:** "Obsidian Glass" — deep, pure blacks ( `#000000` ) mixed with translucent, blurred surface layers ( `bg-white/5 backdrop-blur-xl` ).
- **Typography:** Introduction of a premium sans-serif stack (e.g., Geist or Inter Tight) with high-contrast tracking for metrics.
- **Accents:** Semantic glowing gradients. A location in a heatwave gets subtle amber/crimson background glows; a freezing location gets cyan/blue glows.

### 2.2 Advanced Layout: Bento Grid

---

Move away from traditional stacked columns to a **Bento Grid** architecture. - The dashboard will resemble a highly organized control panel. - KPIs, charts, and file lists will snap into proportional grid cells that animate upon resizing.

### 2.3 Fluid Micro-Animations ( `framer-motion` )

---

- **State Transitions:** Replace sudden component mounting with `AnimatePresence` cross-fades.
  - **Interactions:** Buttons that gently scale down on press ( `whileTap={{ scale: 0.98 }}` ), and cards that lift on hover with dynamic shadow casting.
  - **Loading:** Replace the static spinner with cascading shimmer effects (Skeleton loaders) that map to the exact shape of the expected data.
- 

## 3. Top-Level Feature Set

---

## Feature 1: Semantic Geocoding Search (No more Lat/Lon math)

---

**Current:** User must google latitudes and longitudes. **Upgrade:** Integrate the **Open-Meteo Geocoding API**. The user types "Tokyo", and an animated dropdown provides auto-complete suggestions. Upon selection, the app automatically maps the coordinates and fetches the data.

## Feature 2: Automated Climate Insight KPIs

---

**Current:** The user must read the chart to deduce insights. **Upgrade:** The frontend parses the `DailyWeatherData` and renders "Insight Cards": - 🌡️ **Absolute Maximum:** The highest temperature in the range + the exact date it occurred. - 📉 **Temperature Variance:** The average swing between daily high and low. - ❄️ **Freeze Warning:** Automatic flag if temperatures dropped below 0°C during the period.

## Feature 3: Interactive Multi-Axis Visualizations

---

**Current:** Single line chart. **Upgrade:** Enhance `Recharts` to support custom brush controls (allowing users to zoom into specific date ranges on the chart). Add synced crosshairs that follow the cursor across multiple charts if we expand to precipitation.

## Feature 4: Data Portability (CSV Export)

---

**Current:** Data is locked in the browser table. **Upgrade:** Client-side CSV generation. A "Download Report" button that instantly converts the JSON payload into a downloadable `.csv` file for Excel/Python analysis by climate scientists.

## Feature 5: Toast Notifications

---

**Current:** Inline error text. **Upgrade:** Implement `sonner` or `react-hot-toast` for sleek, non-blocking notification popups at the bottom of the screen for success/error states.

---

## 4. Technical Feasibility & Impact

---

| Upgrade                 | Library Required              | Complexity | Impact                                   |
|-------------------------|-------------------------------|------------|------------------------------------------|
| Geocoding Search        | Native Fetch (Open-Meteo API) | Medium     | 🔥 🔥 🔥 High (Massive UX friction removed) |
| Bento Layout / Glass UI | Tailwind CSS                  | Low        | 🔥 🔥 High (Premium feel)                  |
| Fluid Animations        | <code>framer-motion</code>    | Medium     | 🔥 🔥 High (Application feels "alive")     |
| Insight KPIs            | Custom TS Logic               | Low        | 🔥 🔥 High (Adds actual business value)    |
| CSV Export              | Native JS (Blob)              | Low        | 🔥 Medium (Pro-user feature)              |

## 5. Conclusion

By injecting `framer-motion`, leveraging the free Open-Meteo Geocoding API, and rethinking the data presentation through a Bento Grid of automated insights, the platform will immediately jump from a "coding assignment" to a "SaaS-ready product demo."